POMPS Presentation: 1.1.124

Subhodeep Chakraborty
ee25btech11055,
IIT Hyderabad.

March 31, 2026

Outline

1. Problem Statement
2. Resolving the System
3. Part A: Solving for $G(s)$
4. Part B: Steady-State Analysis
5. Part C: Simulation & Discretization
6. Python Codes
7. Simulation Results

Original Question I

1.1.124 The state space equation of a system is described by (EE 2008)

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \tag{1.1}$$

$$\mathbf{y} = \mathbf{C}\mathbf{x} \tag{1.2}$$

where $\mathbf{x}$ is state vector, $\mathbf{u}$ is input, $\mathbf{y}$ is output and

$$\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 0 & -2 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 \end{pmatrix}.$$

a) The transfer function $G(s)$ of this system will be

i) $\dfrac{s}{(s+2)}$ ii) $\dfrac{s+1}{s(s-2)}$ iii) $\dfrac{s}{(s-2)}$ iv) $\dfrac{1}{s(s+2)}$

Original Question II

b) A unity feedback is provided to the above system $G(s)$ to make it a closed-loop system as shown.

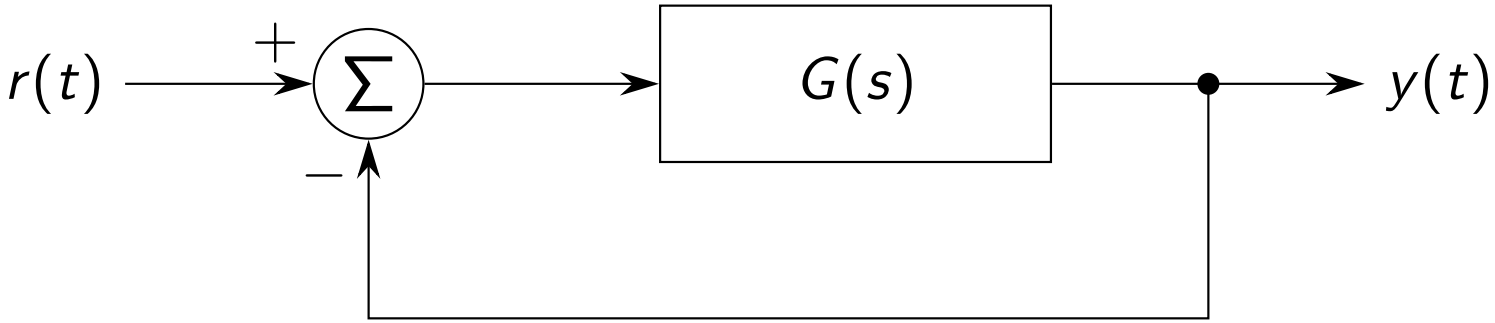


Fig. 1.1.124.1

For a unit step input $r(t)$, the steady state error in the output will be

i) 0 ii) 1 iii) 2 iv) ∞

State-Space Representation

State-Space simplifies high-order DEs for representing systems by breaking any n-th order equation into n coupled 1st-order equations:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}u(t) \quad \text{(State Equation)} \tag{2.1}$$

$$y(t) = \mathbf{C}\mathbf{x}(t) \quad \text{(Output Equation)} \tag{2.2}$$

Deriving the System ODE

By expanding the matrices into algebraic equations:

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} u \implies \begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -2x_2 + u \end{cases} \tag{2.3}$$

$$\text{Applying output equation:} \tag{2.4}$$

$$y = x_1 \tag{2.5}$$

$$\dot{y} = \dot{x}_1 = x_2 \tag{2.6}$$

$$\ddot{y} = \dot{x}_2 \tag{2.7}$$

$$\implies \ddot{y} = -2x_2 + u \tag{2.8}$$

$$\ddot{y} = -2\dot{y} + u \implies \ddot{y} + 2\dot{y} = u(t) \tag{2.9}$$

Deriving Transfer Function

- Frequency-domain transfer function is given by $G(s) = \frac{Y(s)}{U(s)}$
- Apply the Laplace transform to the state equation, assuming zero initial conditions ($\mathbf{x}(0) = 0$):

$$\mathcal{L}\{\dot{\mathbf{x}}(t)\} = \mathcal{L}\{\mathbf{A}\mathbf{x}(t) + \mathbf{B}u(t)\} \tag{2.10}$$

$$s\mathbf{X}(s) = \mathbf{A}\mathbf{X}(s) + \mathbf{B}U(s) \tag{2.11}$$

$$\implies (s\mathbf{I} - \mathbf{A})\mathbf{X}(s) = \mathbf{B}U(s) \implies \mathbf{X}(s) = (s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}U(s) \tag{2.12}$$

$$\implies Y(s) = \mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}U(s) \implies G(s) = \mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} \tag{2.13}$$

Direct Resolution

- Using the derived formula $G(s) = \mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}$.
- First, compute Resolvent Matrix: $(s\mathbf{I} - \mathbf{A})^{-1}$:

$$s\mathbf{I} - \mathbf{A} = \begin{pmatrix} s & -1 \\ 0 & s+2 \end{pmatrix} \tag{3.1}$$

$$(s\mathbf{I} - \mathbf{A})^{-1} = \frac{1}{s(s+2)} \begin{pmatrix} s+2 & 1 \\ 0 & s \end{pmatrix} \tag{3.2}$$

- Multiplying by the input and output vectors:

$$G(s) = \begin{pmatrix} 1 & 0 \end{pmatrix} \left[\frac{1}{s(s+2)} \begin{pmatrix} s+2 & 1 \\ 0 & s \end{pmatrix} \right] \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \frac{1}{s(s+2)} \tag{3.3}$$

Modal Analysis

- Looking at $\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 0 & -2 \end{pmatrix}$. The top row gives $\dot{x}_1 = x_2$. The states are coupled together, which is why direct matrix inversion can be tedious for large systems.
- Thus, we use Spectral Decomposition to find a new coordinate system where the states are independent. This breaks the complex system into simple, parallel first-order filters.
- We do this by finding the eigenvalues and eigenvectors of $\mathbf{A}$.

$$\det(\lambda \mathbf{I} - \mathbf{A}) = \lambda(\lambda + 2) = 0 \implies \lambda_1 = 0, \lambda_2 = -2 \tag{3.4}$$

Modal Analysis

- Using the eigenvectors, we construct the modal matrix $\mathbf{V}$ and its inverse $\mathbf{V}^{-1}$:

$$\mathbf{V} = \begin{pmatrix} 1 & 1 \\ 0 & -2 \end{pmatrix}, \quad \mathbf{V}^{-1} = \begin{pmatrix} 1 & 0.5 \\ 0 & -0.5 \end{pmatrix} \tag{3.5}$$

- We transform the system: $\mathbf{\Lambda} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}$, $\mathbf{B}_m = \mathbf{V}^{-1}\mathbf{B}$, $\mathbf{C}_m = \mathbf{C}\mathbf{V}$.
- The transformed matrix $\mathbf{\Lambda}$ is perfectly diagonal:

$$\mathbf{\Lambda} = \begin{pmatrix} 0 & 0 \\ 0 & -2 \end{pmatrix}, \quad \mathbf{B}_m = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix}, \quad \mathbf{C}_m = \begin{pmatrix} 1 & 1 \end{pmatrix} \tag{3.6}$$

- Because $\mathbf{\Lambda}$ is diagonal, $(s\mathbf{I} - \mathbf{\Lambda})^{-1}$ is simply the inverted diagonal entries:

$$G(s) = \mathbf{C}_m(s\mathbf{I} - \mathbf{\Lambda})^{-1}\mathbf{B}_m = \frac{0.5}{s} - \frac{0.5}{s+2} = \frac{1}{s(s+2)} \tag{3.7}$$

Deriving the Closed-Loop Matrix ($\mathbf{A}_{cl}$)

- Unity negative feedback means the input $u(t)$ is the difference between the reference $r(t)$ and output $y(t)$:

$$u(t) = r(t) - y(t) = r(t) - \mathbf{C}\mathbf{x}(t) \tag{4.1}$$

- Substitute this back into the original state equation:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}(r(t) - \mathbf{C}\mathbf{x}(t)) \tag{4.2}$$

$$\dot{\mathbf{x}}(t) = (\mathbf{A} - \mathbf{B}\mathbf{C})\mathbf{x}(t) + \mathbf{B}r(t) \tag{4.3}$$

- The new system dynamics are governed by $\mathbf{A}_{cl} = \mathbf{A} - \mathbf{B}\mathbf{C}$:

$$\mathbf{A}_{cl} = \begin{pmatrix} 0 & 1 \\ 0 & -2 \end{pmatrix} - \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & -2 \end{pmatrix} \tag{4.4}$$

Equivalence of Time and Frequency Limits

- To find the steady-state output y_{ss}, we can evaluate the system as $t \to \infty$ or as $s \to 0$. They are physically identical.
- **Time Domain ($t \to \infty$):** The system settles, so velocity $\dot{\mathbf{x}} = \mathbf{0}$.

$$\mathbf{0} = \mathbf{A}_{cl}\mathbf{x}_{ss} + \mathbf{B}r_{ss} \implies \mathbf{x}_{ss} = -\mathbf{A}_{cl}^{-1}\mathbf{B}r_{ss} \tag{4.5}$$

$$y_{ss} = \mathbf{C}\mathbf{x}_{ss} = \left(-\mathbf{C}\mathbf{A}_{cl}^{-1}\mathbf{B}\right) r_{ss} \tag{4.6}$$

- **Frequency Domain ($s \to 0$):** DC Gain $H(0)$. Let $s = 0$ in the closed-loop transfer function $H(s) = \mathbf{C}(s\mathbf{I} - \mathbf{A}_{cl})^{-1}\mathbf{B}$:

$$H(0) = -\mathbf{C}\mathbf{A}_{cl}^{-1}\mathbf{B} \implies y_{ss} = H(0)r_{ss} \tag{4.7}$$

Calculating Steady-State Error (e_{ss})

- Calculate the unified DC gain factor, $-\mathbf{C}\mathbf{A}_{cl}^{-1}\mathbf{B}$:

$$\mathbf{A}_{cl}^{-1} = \frac{1}{(0)(-2)-(-1)(1)}\begin{pmatrix}-2 & -1\\ 1 & 0\end{pmatrix} = \begin{pmatrix}-2 & -1\\ 1 & 0\end{pmatrix} \tag{4.8}$$

$$H(0) = -\begin{pmatrix}1 & 0\end{pmatrix}\begin{pmatrix}-2 & -1\\ 1 & 0\end{pmatrix}\begin{pmatrix}0\\ 1\end{pmatrix} = -(-1) = 1 \tag{4.9}$$

- For a unit step input ($r_{ss} = 1$), the output is $y_{ss} = 1 \times 1 = 1$.
- The error is defined as $e_{ss} = \text{Input} - \text{Output}$:

$$e_{ss} = 1 - 1 = 0 \tag{4.10}$$

Transient Analysis: Critical Damping

- Before simulating the transition to steady-state, the roots of the closed-loop matrix $\mathbf{A}_{cl}$ are given by:

$$\det(\lambda \mathbf{I} - \mathbf{A}_{cl}) = \lambda^2 + 2\lambda + 1 = (\lambda + 1)^2 = 0 \tag{5.1}$$

- The system has a repeated real root at $\lambda = -1$.
- Comparing with the standard 2nd order DE $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$:

$$\omega_n^2 = 1 \implies \omega_n = 1 \tag{5.2}$$

$$2\zeta\omega_n = 2 \implies \zeta = 1 \tag{5.3}$$

- $\zeta = 1$ dictates the system is critically damped.

Method 1: Numerical Integration via RK4

- At step t_n with size h, RK4 averages four intermediate slopes:

$$\mathbf{k}_1 = \mathbf{A}_{cl}\mathbf{x}_n + \mathbf{B}r(t_n) \tag{5.4}$$

$$\mathbf{k}_2 = \mathbf{A}_{cl}\left(\mathbf{x}_n + \frac{h}{2}\mathbf{k}_1\right) + \mathbf{B}r\left(t_n + \frac{h}{2}\right) \tag{5.5}$$

$$\mathbf{k}_3 = \mathbf{A}_{cl}\left(\mathbf{x}_n + \frac{h}{2}\mathbf{k}_2\right) + \mathbf{B}r\left(t_n + \frac{h}{2}\right) \tag{5.6}$$

$$\mathbf{k}_4 = \mathbf{A}_{cl}(\mathbf{x}_n + h\mathbf{k}_3) + \mathbf{B}r(t_n + h) \tag{5.7}$$

- State update: $\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$.
- However, this approximation has $\mathcal{O}(h^5)$ error. RK4 is normally used when shape of graph is unknown, for LTI system it's redundant.

Method 2: Deriving Discrete Difference Equation

- Instead of guessing the slope, we start with the exact analytical continuous solution to $\dot{\mathbf{x}}(t) = \mathbf{A}_{cl}\mathbf{x}(t) + \mathbf{B}r(t)$:

$$\mathbf{x}(t) = e^{\mathbf{A}_{cl}(t-t_0)}\mathbf{x}(t_0) + \int_{t_0}^{t} e^{\mathbf{A}_{cl}(t-\tau)}\mathbf{B}r(\tau)d\tau \tag{5.8}$$

- To compute the next digital sample $\mathbf{x}[k+1]$ at time $t = (k+1)T_s$ using the current sample $\mathbf{x}[k]$ at $t_0 = kT_s$, we substitute these times into the integral:

$$\mathbf{x}((k+1)T_s) = e^{\mathbf{A}_{cl}T_s}\mathbf{x}(kT_s) + \int_{kT_s}^{(k+1)T_s} e^{\mathbf{A}_{cl}((k+1)T_s-\tau)}\mathbf{B}r(\tau)d\tau \tag{5.9}$$

Method 2: Deriving Discrete Difference Equation

- Usually, the input $r(\tau)$ is held constant during the sample interval. So, $r(\tau) = r[k]$.
- Because $r[k]$ is constant, we can pull it completely out of the integral.
- Let $\sigma = (k+1)T_s - \tau$:

$$\mathbf{x}[k+1] = \underbrace{\left(e^{\mathbf{A}_{cl}T_s}\right)}_{\mathbf{\Phi}_d}\mathbf{x}[k] + \underbrace{\left(\int_0^{T_s} e^{\mathbf{A}_{cl}\sigma}d\sigma\mathbf{B}\right)}_{\mathbf{\Gamma}_d} r[k] \tag{5.10}$$

- Thus the exact discrete difference equation is $\mathbf{x}[k+1] = \mathbf{\Phi}_d\mathbf{x}[k] + \mathbf{\Gamma}_d r[k]$.

Method 2: Solving the Matrix Exponential

- To use this exact mapping, we must evaluate $\mathbf{\Phi}_d = e^{\mathbf{A}_{cl} T_s}$.
- Because our system is critically damped (repeated eigenvalue $\lambda = -1$), the **Cayley-Hamilton Theorem** allows us to evaluate the infinite Taylor series of the exponential exactly as a simple linear polynomial:

$$e^{\mathbf{A}_{cl} t} = e^{\lambda t} \left(\mathbf{I} + (\mathbf{A}_{cl} - \lambda \mathbf{I}) t \right) \tag{5.11}$$

$$\mathbf{\Phi}_d = e^{-T_s} \begin{pmatrix} 1 + T_s & T_s \\ -T_s & 1 - T_s \end{pmatrix} \tag{5.12}$$

- Using these exact discrete matrices ensures that our step-by-step software simulations will flawlessly mimic the physical analog reality without any integration drift.

Python Implementation I

```python
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal
import sympy as sp
import os

# Create the target directory for figures
os.makedirs('../figs', exist_ok=True)

# ==========================================
# PART 1: Symbolic Verification (SymPy)
# ==========================================
print("--- Symbolic Verification ---")

# Define symbols and matrices
s, t = sp.symbols('s t')
A_sym = sp.Matrix([[0, 1], [0, -2]])
B_sym = sp.Matrix([[0], [1]])
C_sym = sp.Matrix([[1, 0]])
I_sym = sp.eye(2)

# 1. Verify Transfer Function G(s) = C(sI - A)^-1 B
```

Python Implementation II

```python
G_s = C_sym * (s * I_sym - A_sym).inv() * B_sym
G_s_simplified = sp.simplify(G_s[0])
print(f"1. Open-Loop Transfer Function G(s): {G_s_simplified}")

# 2. Verify Closed-Loop Matrix A_cl = A - BC
A_cl_sym = A_sym - B_sym * C_sym
print(f"2. Closed-Loop Matrix A_cl:\n{np.array(A_cl_sym.tolist())}")

# 3. Verify Exact Discretization Matrix (Matrix Exponential Phi_d)
Phi_d_sym = sp.simplify(sp.exp(A_cl_sym * t))
print(f"3. Matrix Exponential Phi_d(t) = e^(A_cl*t):\n{Phi_d_sym}")
print("-" * 40)


# ==========================================
# PART 2: Numerical System Definition
# ==========================================
print("--- Running Numerical Simulations ---")
# Explicitly float type to avoid internal truncation
A = np.array([[0, 1], [0, -2]], dtype=float)
B = np.array([[0], [1]], dtype=float)
C = np.array([[1, 0]], dtype=float)
D = np.array([[0]], dtype=float)
```

Python Implementation III

```python
A_cl = A - B @ C

Ts = 0.5
t_end = 8.0
t_samples = np.arange(0, t_end + Ts, Ts)


# ==========================================
# PART 3: Pure Analytical Continuous Solution
# ==========================================
# Using the exact mathematical derivation y(t) = 1 - (1+t)*e^(-t)
def exact_continuous(t):
    return 1.0 - (1.0 + t) * np.exp(-t)

# Smooth continuous curves for plotting
t_cont = np.linspace(0, t_end, 500)
y_cont = exact_continuous(t_cont)

t_dense = np.linspace(1.48, 1.52, 1000)
y_dense = exact_continuous(t_dense)

# True values exactly at sample times (for error calculation)
y_true_samples = exact_continuous(t_samples)
```

Python Implementation IV

```python


# ==========================================
# PART 4: Exact Discrete Method (Matrix Exponential)
# ==========================================
sys_cl_disc = signal.cont2discrete((A_cl, B, C, D), Ts, method='zoh')
_, y_disc = signal.dstep(sys_cl_disc, t=t_samples)
y_disc = np.squeeze(y_disc)


# ==========================================
# PART 5: RK4 Integration Method
# ==========================================
def rk4_step_response(A, B, C, Ts, t_end):
    t = np.arange(0, t_end + Ts, Ts)
    x = np.zeros((2, len(t)))
    for i in range(len(t) - 1):
        xi = x[:, i:i+1]
        k1 = A @ xi + B * 1.0
        k2 = A @ (xi + (Ts / 2) * k1) + B * 1.0
        k3 = A @ (xi + (Ts / 2) * k2) + B * 1.0
        k4 = A @ (xi + Ts * k3) + B * 1.0
        x[:, i+1:i+2] = xi + (Ts / 6) * (k1 + 2*k2 + 2*k3 + k4)
```

Python Implementation V

```
    return t, (C @ x).flatten()

_, y_rk4 = rk4_step_response(A_cl, B, C, Ts, t_end)


# ==========================================
# PART 6: Error Calculations
# ==========================================
# Add an infinitesimally small number (1e-20) to avoid log(0) errors
error_disc = np.abs(y_true_samples - y_disc) + 1e-20
error_rk4 = np.abs(y_true_samples - y_rk4) + 1e-20


# ==========================================
# PART 7: Plotting and Saving Figures
# ==========================================

# --- FIGURE 1: Exact Discrete ---
plt.figure(figsize=(10, 6))
plt.plot(t_cont, y_cont, 'b-', linewidth=2, label='Continuous Analytical Curve')
plt.step(t_samples, y_disc, 'ro--', where='post', label=f'Exact Discrete ZOH (
    $T_s$={Ts}s)')
```

Python Implementation VI

```
plt.axhline(1.0, color='k', linestyle=':', linewidth=2, label='Reference $r(t) = 1$')
plt.title('Figure 1: Continuous Analytical vs. Exact Discrete Method', fontsize=14)
plt.xlabel('Time (seconds)', fontsize=12)
plt.ylabel('Output $y(t)$', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(loc='lower right', fontsize=11)
plt.tight_layout()
plt.savefig('../figs/fig1_exact_discrete.png', dpi=300)
plt.close()

# --- FIGURE 2: RK4 Approximation ---
plt.figure(figsize=(10, 6))
plt.plot(t_cont, y_cont, 'b-', linewidth=2, label='Continuous Analytical Curve')
plt.plot(t_samples, y_rk4, 'gs-', markersize=6, alpha=0.8, label=f'RK4 Approximation ($h$={Ts}s)')
plt.axhline(1.0, color='k', linestyle=':', linewidth=2, label='Reference $r(t) = 1$')
plt.title('Figure 2: Continuous Analytical vs. RK4 Integration', fontsize=14)
plt.xlabel('Time (seconds)', fontsize=12)
plt.ylabel('Output $y(t)$', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
```

Python Implementation VII

```
plt.legend(loc='lower right', fontsize=11)
plt.tight_layout()
plt.savefig('../figs/fig2_rk4_comparison.png', dpi=300)
plt.close()

# --- FIGURE 3: Microscopic Zoom at t = 1.5s ---
plt.figure(figsize=(10, 6))
plt.plot(t_dense, y_dense, 'b-', linewidth=2, label='Continuous Analytical Curve')
idx = 3 # Index corresponding to t=1.5s
plt.plot(t_samples[idx], y_disc[idx], 'ro', markersize=10, label='Exact Discrete Point')
plt.plot(t_samples[idx], y_rk4[idx], 'gs', markersize=10, label='RK4 Point')
plt.title('Figure 3: Microscopic Zoom at $t = 1.5$s', fontsize=14)
plt.xlabel('Time (seconds)', fontsize=12)
plt.ylabel('Output $y(t)$', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(loc='lower right', fontsize=11)
plt.tight_layout()
plt.savefig('../figs/fig3_zoomed.png', dpi=300)
plt.close()

# --- FIGURE 4: Error Magnitude (Log Scale) ---
```

Python Implementation VIII

```
plt.figure(figsize=(10, 6))
plt.semilogy(t_samples, error_disc, 'ro-', linewidth=2, label='Exact Discrete
    Error')
plt.semilogy(t_samples, error_rk4, 'gs-', linewidth=2, label='RK4 Truncation
    Error')
plt.axhline(1e-15, color='k', linestyle=':', linewidth=2, label='Machine
    Precision (~1e-15)')
plt.title('Figure 4: Absolute Error vs. True Analytical Solution', fontsize=14)
plt.xlabel('Time (seconds)', fontsize=12)
plt.ylabel('Absolute Error $|y_{true} - y_{sim}|$ (Log Scale)', fontsize=12)
plt.grid(True, which="both", linestyle='--', alpha=0.5)
plt.legend(loc='center right', fontsize=11)
plt.tight_layout()
plt.savefig('../figs/fig4_error_log.png', dpi=300)
plt.close()

#print("All 4 figures generated and saved successfully!")
```

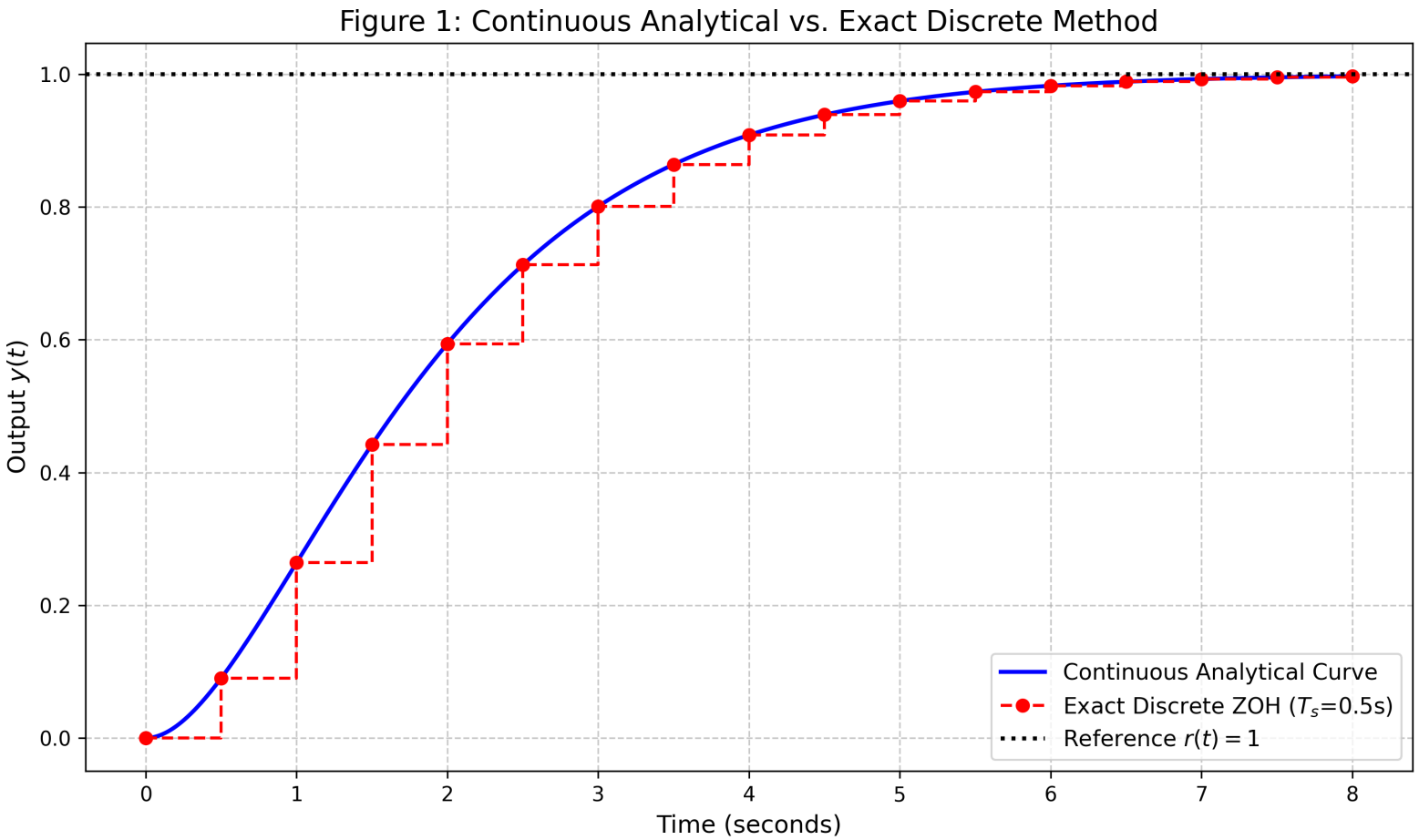
Sympy Verification

- Sympy confirms all of the expressions we derived. Computer algebra systems are extremely cool.

```
--- Symbolic Verification ---
1. Open-Loop Transfer Function G(s): 1/(s*(s + 2))
2. Closed-Loop Matrix A_cl:
[[0 1]
 [-1 -2]]
3. Matrix Exponential Phi_d(t) = e^(A_cl*t):
Matrix([[(t + 1)*exp(-t), t*exp(-t)], [-t*exp(-t), (1 - t)*exp(-t)]])
----------------------------------------
```

Simulation: Exact Discrete Method

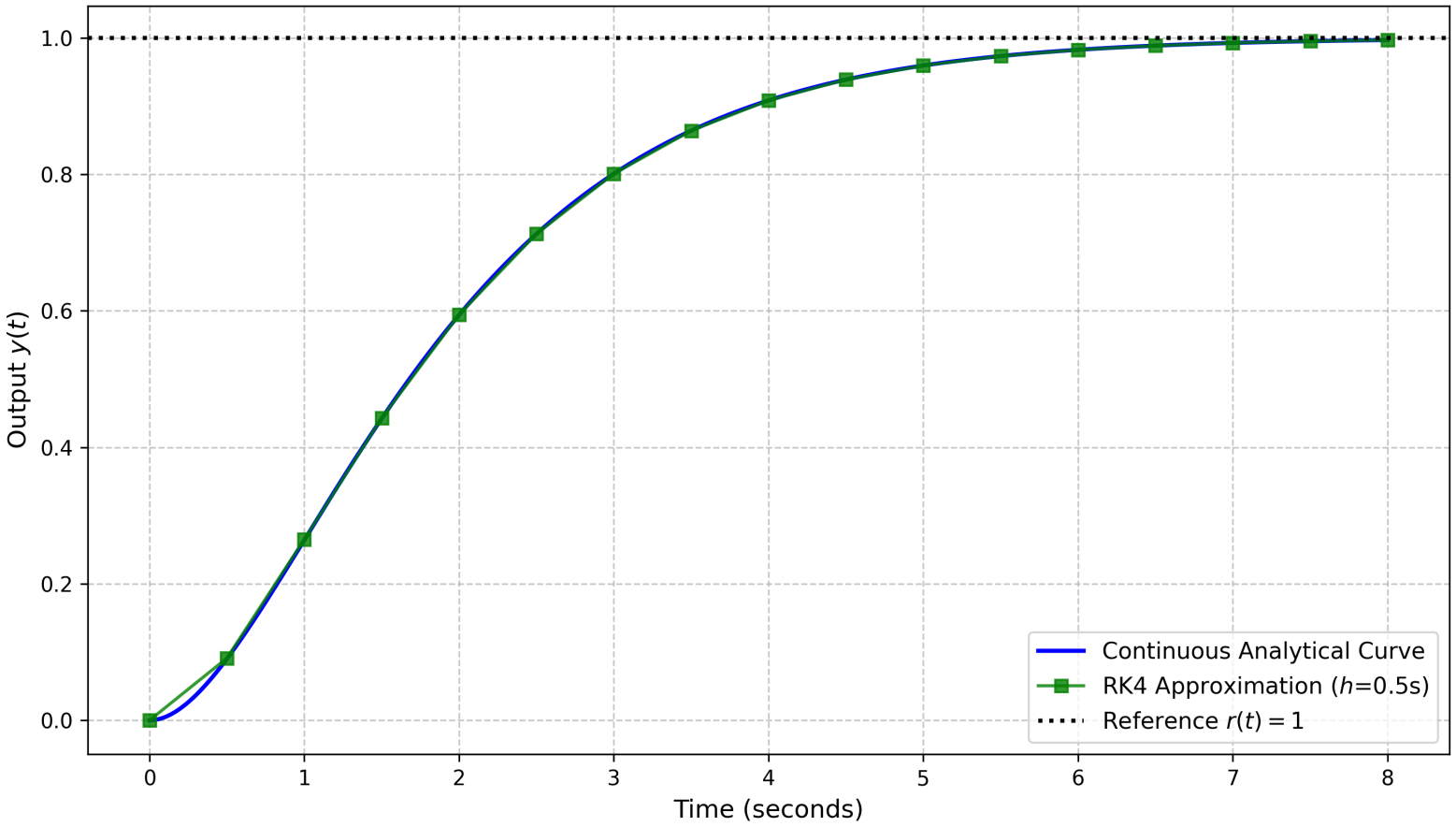
- Using our derived $\mathbf{\Phi}_d$ and $\mathbf{\Gamma}_d$, the digital simulation lands precisely on the continuous analytical curve at every sample instant ($T_s = 0.5$s).



Simulation: RK4 Integration Approximation

- Using the 4th-Order Runge-Kutta method with the same step size ($h = 0.5$s).
- On a macro scale, numerical integration appears to track the system perfectly toward the steady-state of 1.

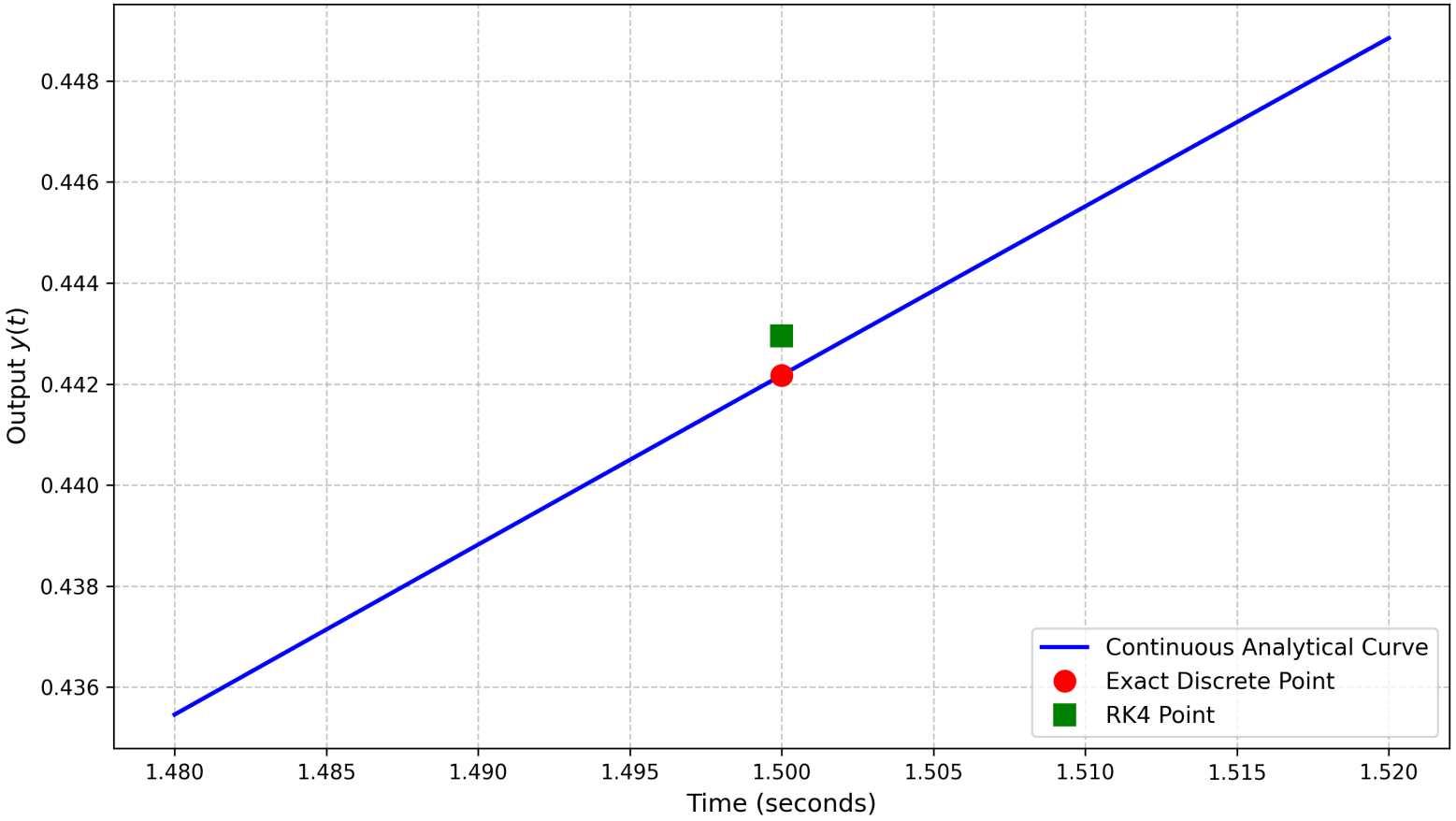
Figure 2: Continuous Analytical vs. RK4 Integration



Analysis at a Point

- Zooming in precisely at $t = 1.5$s exposes the underlying geometry.
- Because RK4 is approximating the slope rather than using the exact analytical mapping, it misses the true curve.
- The exact discrete method computes the state flawlessly.

Figure 3: Microscopic Zoom at $t = 1.5$s



Error Analysis

- Plotting the absolute error $|y_{true} - y_{sim}|$ on a logarithmic scale.
- RK4 produces an error hovering between 10^{-3} and 10^{-5}.
- The Exact Discrete formulation's error is strictly bounded by IEEE 754 float limitations ($\sim 10^{-15}$), making it mathematically exact.

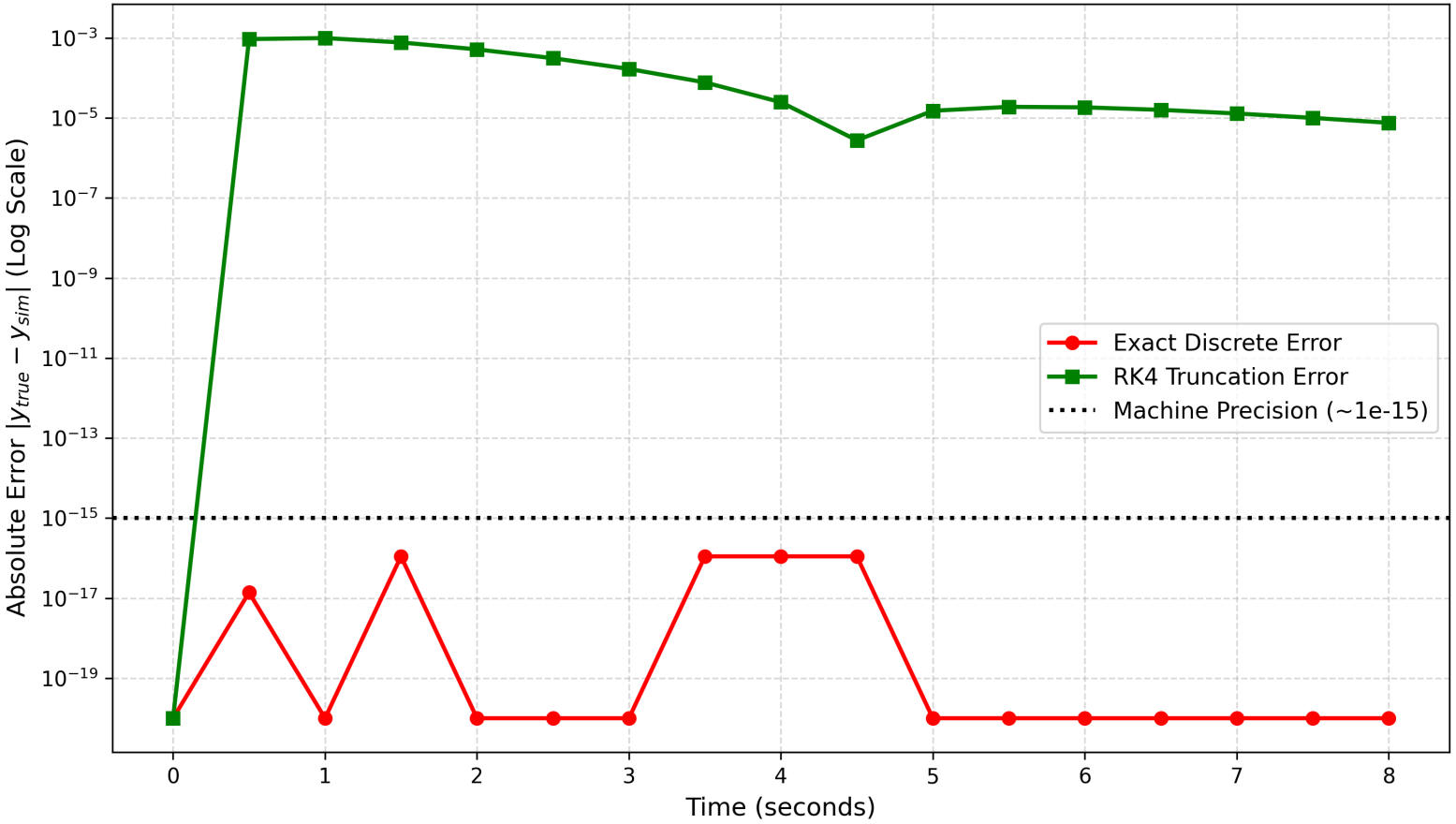



Figure 4: Absolute Error vs. True Analytical Solution